COS 433: Cryptography

Announcements:

- Problem Set 3 due **10/28**

- Lecture notes on computational number theory (by Dana Angluin) on the course website

Factoring and Cryptography

Factoring Assumption

Assumption (Hardness of Factoring)

For all poly-time A,

$$\Pr_{\substack{p,q \leftarrow \{0,1\}^\lambda \\ p,q \text{ prime}}}[A(pq) = (p,q)] = \mathrm{negl}(\lambda)$$

Theorem: under the factoring assumption, OWFs exist

Theorem:

Factoring $N \Leftrightarrow$ Solving quadratic equations over $\mathbb{Z}_N$

Square root mod N assumption

Assumption:

For all poly-time A,

$$\Pr_{\substack{p,q \leftarrow \{0,1\}^\lambda \\ p,q \text{ prime}, N = pq \\ x \leftarrow \mathbb{Z}_N}} [A(N, x^2 \bmod N) = y : y^2 \equiv x^2 \bmod N] = \text{negl}(\lambda)$$

Theorem:

Factoring assumption $\Leftrightarrow$ Square root mod N assumption

OWFs based on Factoring

We believe this (modulo domain issues) is a OWF:

$$f(p, q) = p \cdot q$$

OWFs based on Factoring

Here's another candidate OWF (also secure under factoring assumption):

$$f(p, q, x) = (pq, x^2 \bmod pq)$$

Can't even find a square root x, let alone (p, q) (this is secretly very cool)

Question: is this a "PRG?"

$$f(p, q, x) \approx_c (pq, u \bmod pq)?$$

Pseudorandomness based on Factoring?

Question: is it the case that

$$(pq, x^2 \bmod pq) \approx_c (pq, u \bmod pq)?$$

Answer: no!

- u is a square modulo p, q with probability ½ (independently)
- Efficient algorithm calculating parity of these two bits of info

Pseudorandomness based on Factoring?

Definition: the Jacobi symbol

For $N = p_1 \cdot \ldots \cdot p_k$, and $\left(\frac{a}{N}\right) = \left(\frac{a}{p_1}\right) \cdot \ldots \cdot \left(\frac{a}{p_k}\right)$

$\left(\frac{a}{p}\right) = \pm 1$ depending on whether a is a square mod p

Fact: if a is a square mod N, then $\left(\frac{a}{N}\right) = 1$.

Fact: $\mathbf{E}_{u \leftarrow \mathbb{Z}_N^\times} \left(\frac{u}{N}\right) = 0$

Pseudorandomness based on Factoring?

Definition: the Jacobi symbol

For $N = p_1 \cdot \ldots \cdot p_k$, and $\left(\frac{a}{N}\right) = \left(\frac{a}{p_1}\right) \cdot \ldots \cdot \left(\frac{a}{p_k}\right)$

$\left(\frac{a}{p}\right) = \pm 1$ depending on whether a is a square mod p

Crazy Fact: $\left(\frac{a}{N}\right)$ is efficiently computable given (a, N)

Reason: for odd (a, N): $\left(\frac{a}{N}\right) = (-1)^{\frac{a-1}{2} \cdot \frac{N-1}{2}} \cdot \left(\frac{N}{a}\right)$ (quadratic reciprocity)

Pseudorandomness based on Factoring?

Question: is it the case that

$$(pq, x^2 \bmod pq) \approx_c (pq, u \bmod pq)?$$

Answer: no!

But this is the only known ”obstacle” to pseudorandomness.

Pseudorandomness based on Factoring?

Assumption (Quadratic Residuosity):

$$(pq, x^2 \bmod pq) \approx_c (pq, u \bmod pq)$$

Where u is a uniform element of $\mathbb{Z}_N^\times$ subject to the condition that

$$\left(\frac{u}{N}\right) = 1$$

Pseudorandomness based on Factoring?

Assumption (Quadratic Residuosity, equivalent formulation):

$$(pq, x^2 \bmod pq) \approx_c (pq, a \cdot x^2 \bmod pq)$$

Where a is any fixed element of $\mathbb{Z}_N^\times$ such that $\left(\frac{a}{N}\right) = 1$

Equivalent because:

- if a is a square, $a \cdot x^2$ is a random square.
- if a is not a square, $a \cdot x^2$ is uniform non-square with Jacobi symbol 1

Pseudorandomness based on Factoring?

Assumption (Quadratic Residuosity, equivalent formulation):

$$(pq, x^2 \bmod pq) \approx_c (pq, a \cdot x^2 \bmod pq)$$

Where a is any fixed element of $\mathbb{Z}_N^\times$ such that $\left(\frac{a}{N}\right) = 1$

Example:

- if $p, q \equiv 3 \pmod 4$, $a = -1$ is a non-square modulo p and q

Pseudorandomness based on Factoring?

Assumption (Quadratic Residuosity, equivalent formulation):

$$(pq, x^2 \bmod pq) \approx_c (pq, -x^2 \bmod pq)$$

Where a is any fixed element of $\mathbb{Z}_N^\times$ such that $\left(\frac{a}{N}\right) = 1$

Example:

- if $p, q \equiv 3 \pmod 4$, $a = -1$ is a non-square modulo p and q
- note: if factoring is hard, then it is still hard subject to $p, q \equiv 3 \pmod 4$

Pseudorandomness based on Factoring?

Question: Are the following equivalent?

Factoring assumption $\Leftrightarrow$ QR assumption

Open Problem

Quadratic Residuosity

Lemma (QR worst-case to average-case):

Suppose that $A(N, z)$ violates the QR assumption for a fixed N.
Then, there exists an efficient A' (independent of N) such that:

$A'(N, z)$ decides whether **any given** z is a square modulo N

Proof:

- Given z, compute $\left(\frac{z}{N}\right)$. Output "non-square" if -1.
- Otherwise, do the following $\mathbf{poly}(\lambda)$ times:
  - Run $A(N, z \cdot x^2)$ for a uniformly random x.
- Output the majority answer of the calls to A.

Quadratic Residuosity

Assumption (Quadratic Residuosity, equivalent formulation):

$$(pq, x^2 \bmod pq) \approx_c (pq, a \cdot x^2 \bmod pq)$$

Where a is any fixed element of $\mathbb{Z}_N^\times$ such that $\left(\frac{a}{N}\right) = 1$

Not exactly a PRG, but we can use this to encrypt messages!

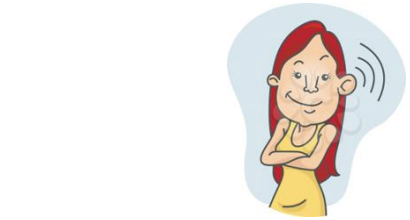
Encryption from QR (Goldwasser-Micali)

Encryption from QR



$k = (p, q)$

$m \in \{+1\}$





$k = (p, q)$

- Key: rand $N = pq$ is a “Blum integer”



Encryption from QR



$k = (p, q)$
$m \in \{\pm 1\}$

$$\mathrm{ct} = r^2 \cdot m \pmod{N}$$





$k = (p, q)$

$$m' = \left(\frac{\mathrm{ct}}{p}\right)$$

- Key: random primes p, q with $p \equiv q \equiv 3 \pmod 4$. $N = pq$ is a "Blum integer"
- To encrypt a one-bit message $m \in \{\pm 1\}$:
  - Sample a random $r \leftarrow \mathbb{Z}_N^\times$.
  - Output $r^2 \cdot m \bmod N$
- To decrypt, check if ct is a square modulo N (equivalently, modulo p or q)

Encryption from QR



$k = (p, q)$
$m \in \{\pm 1\}$

$$\mathrm{ct} = r^2 \cdot m \pmod N$$





$k = (p, q)$

$$m' = \left(\frac{\mathrm{ct}}{p}\right)$$

- Claim: this scheme is CPA-secure!
- One-message security: $r^2 \cdot m \bmod N \approx_c r^2 \cdot m' \bmod N$ by QR.
- Many-message IND-security:
  - Want: $(r_1^2 \cdot m_1 \bmod N, \ldots, r_t^2 \cdot m_t) \bmod N \approx_c (r_1^2 \cdot m_1', \ldots, r_t^2 \cdot m_t') \bmod N$
  - In fact, we claim: $(N, r_1^2 \cdot m_1, \ldots, r_t^2 \cdot m_t) \approx_c (N, r_1^2 \cdot m_1', \ldots, r_t^2 \cdot m_t')$
  - Hybrid argument: change m_i one at a time using QR.

Encryption from QR



$k = (p, q)$
$m \in \{\pm 1\}$

$$\mathrm{ct} = r^2 \cdot m \pmod{N}$$





$k = (p, q)$

$$m' = \left(\frac{\mathrm{ct}}{p}\right)$$

This was really easy! (compared to building/using PRFs)

Why?

Encryption from QR



$k = (p, q)$
$m \in \{\pm 1\}$

$$\mathrm{ct} = r^2 \cdot m \pmod{N}$$





$k = (p, q)$

$$m' = \left(\frac{\mathrm{ct}}{p}\right)$$

Observation 1: Alice only needs N to be able to encrypt! (Not p, q)

Encryption from QR



$N = pq$
$m \in \{\pm 1\}$

$$\mathrm{ct} = r^2 \cdot m \pmod{N}$$





$k = (p, q)$

$$m' = \left(\frac{\mathrm{ct}}{p}\right)$$

Observation 1: Alice only needs N to be able to encrypt! (Not p, q)

Observation 2: secure even if the adversary knows N! (but not p, q)

Encryption from QR



$N = pq$
$m \in \{\pm 1\}$

$$\mathrm{ct} = r^2 \cdot m \pmod{N}$$



$N = pq$



$k = (p, q)$

$$m' = \left(\frac{\mathrm{ct}}{p}\right)$$

Observation 1: Alice only needs N to be able to encrypt! (Not p, q)

Observation 2: secure even if the adversary knows N! (but not p, q)

Encryption from QR



$N = pq$
$m \in \{\pm 1\}$

$$\mathrm{ct} = r^2 \cdot m \pmod{N}$$



$N = pq$



$k = (p, q)$

$$m' = \left(\frac{\mathrm{ct}}{p}\right)$$

In other words: this scheme can have a Public Encryption Key

Encryption from QR





$m \in \{\pm 1\}$

$\mathrm{pk} = N = p \cdot q$ (broadcast)

$\mathrm{ct} = r^2 \cdot m \pmod{N}$



$\mathrm{sk} = (p, q)$

$m' = \left(\frac{\mathrm{ct}}{p}\right)$

In other words: this scheme can have a Public Encryption Key

Encryption from QR





$m \in \{\pm 1\}$

$\mathrm{pk} = N = p \cdot q$ (broadcast)

$\mathrm{ct} = r^2 \cdot m \pmod{N}$



$\mathrm{sk} = (p, q)$

$m' = \left(\frac{\mathrm{ct}}{p}\right)$

No need to share a key in advance!

Public-Key Encryption

Public-Key Encryption: Syntax



$m \in \{0,1\}^\lambda$

pk (broadcast)

$\mathrm{ct} = \mathrm{Enc}(\mathrm{pk}, m; r)$



$(\mathrm{pk}, \mathrm{sk}) \leftarrow \mathrm{Gen}(1^\lambda)$

$m' = \mathrm{Dec}(\mathrm{sk}, \mathrm{ct})$

Definition: A public-key encryption scheme consists of algorithms $(\mathbf{Gen}, \mathbf{Enc}, \mathbf{Dec})$.

- $\mathbf{Gen}(1^\lambda)$: a randomized algorithm that outputs a key pair $(\mathrm{pk}, \mathrm{sk})$
- $\mathbf{Enc}(\mathrm{pk}, m)$: outputs a ciphertext $\mathbf{ct}$ given a public key $\mathbf{pk}$ and message m (randomized)
- $\mathbf{Dec}(\mathrm{sk}, \mathrm{ct})$: outputs a message given a secret key $\mathbf{sk}$ and ciphertext $\mathbf{ct}$

Public-Key Encryption



$m \in \{0,1\}^\lambda$

pk (broadcast)

$\mathrm{ct} = \mathrm{Enc}(\mathrm{pk}, m; r)$



$(\mathrm{pk}, \mathrm{sk}) \leftarrow \mathrm{Gen}(1^\lambda)$

$m' = \mathrm{Dec}(\mathrm{sk}, \mathrm{ct})$

Definition: A public-key encryption scheme consists of algorithms $(\mathbf{Gen}, \mathbf{Enc}, \mathbf{Dec})$. It must satisfy the following properties:

- Efficiency: $\mathbf{Gen}, \mathbf{Enc}, \mathbf{Dec}$ run in polynomial time
- Correctness: $\mathrm{Dec}(\mathrm{sk}, \mathrm{Enc}(\mathrm{pk}, m)) = m$ for all $(\mathrm{pk}, \mathrm{sk}) \leftarrow \mathrm{Gen}(1^\lambda)$.
- Computational security: For all m_0, m_1, $(\mathrm{pk}, \mathrm{Enc}(\mathrm{pk}, m_0)) \approx_c (\mathrm{pk}, \mathrm{Enc}(\mathrm{pk}, m_1))$

Public-Key Encryption



$m \in \{0,1\}^\lambda$

pk (broadcast)

$\text{ct} = \text{Enc}(\text{pk}, m; r)$



$(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$

$m' = \text{Dec}(\text{sk}, \text{ct})$

Theorem (Goldwasser-Micali): under the QR assumption, public-key encryption exists!

Public-Key Encryption: History

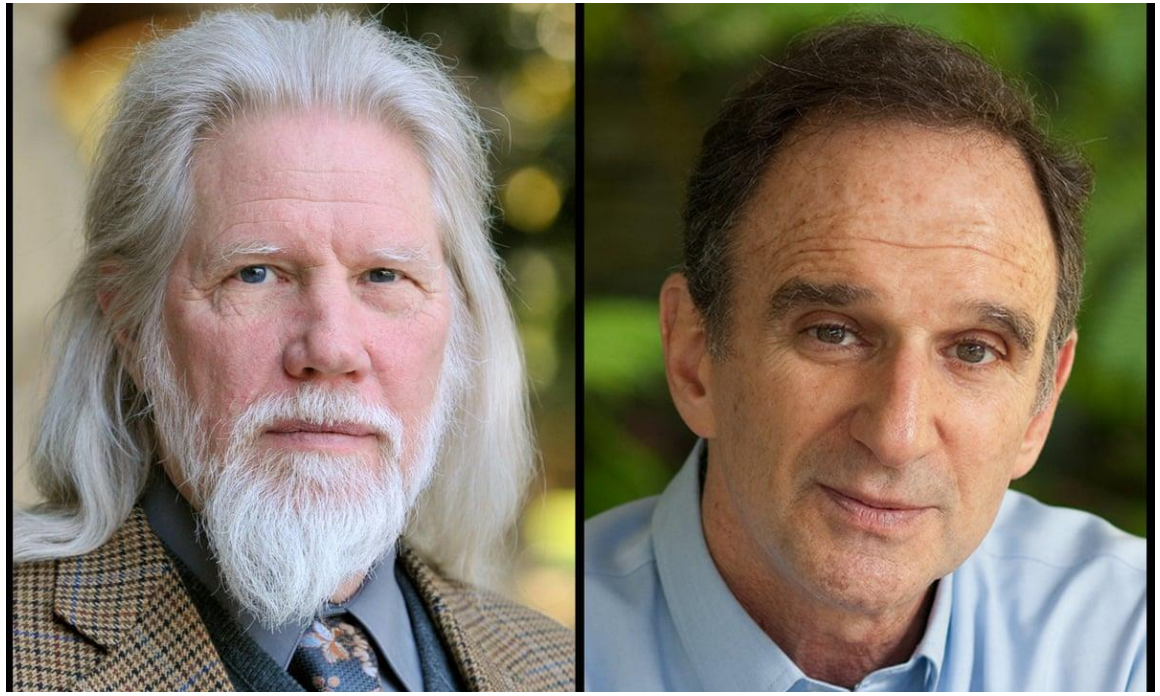
Big problem with SKE: need to meet to agree on a secret key

- What if you can’t/don’t want to fly across the world to agree on a key?
- What if millions of people want to communicate with each other?

Public-Key Encryption: History

Big problem with SKE: need to meet to agree on a secret key

1976: Diffie and Hellman proposed a protocol for secret key agreement



Big problem

1976: Diffie

greement

New Directions in Cryptography

Invited Paper

WHITFIELD DIFFIE AND MARTIN E. HELLMAN, MEMBER, IEEE

***Abstract*—Two kinds of contemporary developments in cryptography are examined. Widening applications of teleprocessing have given rise to a need for new types of cryptographic systems, which minimize the need for secure key distribution channels and supply the equivalent of a written signature. This paper suggests ways to solve these currently open problems. It also discusses how the theories of communication and computation are beginning to provide the tools to solve cryptographic problems of long standing.**

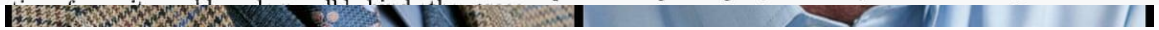
I. Introduction

WE STAND TODAY on the brink of a revolution in cryptography. The development of cheap digital hardware has freed it from the design limitations of mechanical computing and brought the cost of high grade cryptographic devices down to where they can be used in such commercial applications as remote cash dispensers and computer terminals. In turn, such applications create a need for new types of cryptographic systems which minimize the necessity of secure key distribution channels and supply the equivalent of a written signature. At the same time, theoretical developments in information theory and computer science show promise of providing provably secure cryptosystems, changing this ancient art into a science.

The development of computer controlled communication networks promises effortless and inexpensive contact between people or computers on opposite sides of the world, replacing most mail and many excursions with telecommunications. For many applications these contacts must be made secure against both eavesdropping and the injection of illegitimate messages. At present, however, the

The best known cryptographic problem is that of privacy: preventing the unauthorized extraction of information from communications over an insecure channel. In order to use cryptography to insure privacy, however, it is currently necessary for the communicating parties to share a key which is known to no one else. This is done by sending the key in advance over some secure channel such as private courier or registered mail. A private conversation between two people with no prior acquaintance is a common occurrence in business, however, and it is unrealistic to expect initial business contacts to be postponed long enough for keys to be transmitted by some physical means. The cost and delay imposed by this key distribution problem is a major barrier to the transfer of business communications to large teleprocessing networks.

Section III proposes two approaches to transmitting keying information over public (i.e., insecure) channels without compromising the security of the system. In a *public key cryptosystem* enciphering and deciphering are governed by distinct keys, E and D, such that computing D from E is computationally infeasible (e.g., requiring 10^{100} instructions). The enciphering key E can thus be publicly disclosed without compromising the deciphering key D. Each user of the network can, therefore, place his enciphering key in a public directory. This enables any user of the system to send a message to any other user enciphered in such a way that only the intended receiver is able to decipher it. As such, a public key cryptosystem is a multiple access cipher. A private conversation can therefore be held between any two individuals regardless of whether they have ever communicated before. Each one sends messages to the other enciphered in the receiver's public enciphering key and deciphers the messages he re-



Public-Key Encryption: History

Big problem with SKE: need to meet to agree on a secret key

1976: Diffie and Hellman proposed a protocol for secret key agreement

1978: Rivest, Shamir and Adleman propose a practical PKE scheme related to (but different from) Goldwasser-Micali







Public-Key Encryption: History

1976: Diffie and Hellman proposed a protocol for secret key agreement

1978: Rivest, Shamir and Adleman propose a practical PKE scheme related to (but different from) Goldwasser-Micali (and later formed a company)

1982: Goldwasser and Micali defined and constructed computationally secure PKE (randomized encryption + security definition + proof)

Early 1990s: public-key cryptography (RSA encryption, other schemes we will learn about) widely adopted for commercial use.

Public-Key Encryption: History

1976: Diffie and Hellman proposed a protocol for secret key agreement

1978: Rivest, Shamir and Adleman propose a practical PKE scheme related to (but different from) Goldwasser-Micali (and later formed a company)

1997: GCHQ (British intelligence) revealed that it had independently developed RSA encryption between 1970-1974 (Ellis, Cocks, Williamson).

PKE has many nice properties!

Public-Key Encryption (one-bit messages)



$m \in \{0,1\}$

pk (broadcast)

$\mathrm{ct} = \mathrm{Enc}(\mathrm{pk}, m; r)$



$(\mathrm{pk}, \mathrm{sk}) \leftarrow \mathrm{Gen}(1^\lambda)$

$m' = \mathrm{Dec}(\mathrm{sk}, \mathrm{ct})$

Definition: A public-key encryption scheme consists of algorithms $(\mathbf{Gen}, \mathbf{Enc}, \mathbf{Dec})$. It must satisfy the following properties:

- Efficiency: $\mathbf{Gen}, \mathbf{Enc}, \mathbf{Dec}$ run in polynomial time
- Correctness: $\mathrm{Dec}(\mathrm{sk}, \mathrm{Enc}(\mathrm{pk}, m)) = m$ for all $(\mathrm{pk}, \mathrm{sk}) \leftarrow \mathrm{Gen}(1^\lambda)$.
- Computational security: $(\mathrm{pk}, \mathrm{Enc}(\mathrm{pk}, 0)) \approx_c (\mathrm{pk}, \mathrm{Enc}(\mathrm{pk}, 1))$

CPA Security for PKE

Challenger



pk

$b \leftarrow \{0,1\}$

pk

$m_0^{(\lambda)}, m_1^{(\lambda)}$

$\mathrm{Enc}(\mathrm{pk}, m_b; r)$

Adversary



b'

Win if $b' = b$

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies indistinguishability (IND-CPA) security if

For all polynomial-time  ,

$$\Pr[\mathrm{Win}] \leq \frac{1}{2} + \mathrm{negl}(\lambda)$$

CPA Security for PKE (equivalent)

Challenger



pk

$b \leftarrow \{0,1\}$

pk

$m_0^{(\lambda)}, m_1^{(\lambda)}$

$\mathrm{Enc}(\mathrm{pk}, m_b; r)$

Adversary



b'

Win if $b' = b$

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies indistinguishability (IND-CPA) security if

For all polynomial-time  ,

$$\Pr[\mathrm{Win}] \leq \frac{1}{2} + \mathrm{negl}(\lambda)$$

Public-Key Encryption



$m \in \{0,1\}^n$

pk (broadcast) ←

$\text{ct} = \text{Enc}(\text{pk}, m; r)$ →



$(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$

$m' = \text{Dec}(\text{sk}, \text{ct})$

Lemma: Suppose that $(\text{Gen}, \text{Enc}, \text{Dec})$ is a public-key encryption scheme with messages of length $n = 1$. Then, there exists a PKE with message length $n(\lambda)$ for all polynomial functions $n(\lambda)$, satisfying CPA-security.

Proof: encrypt bit-by-bit. Security holds by a hybrid argument.

Next: RSA Encryption

RSA Computational Problem

How hard is it to solve equations in one variable over $\mathbb{Z}_N$?

As hard as factoring: Given a, find x such that $x^2 \equiv a \pmod{N}$

QR (believed hard): Given a, decide if $x^2 \equiv a \pmod{N}$ has a solution

What about equations of higher degree?

RSA Computational Problem

How hard is it to solve equations in one variable over $\mathbb{Z}_N$?

Cube root finding (believed hard): Given a, find x such that $x^3 \equiv a \pmod{N}$

Candidate OWF:

$$f(p, q, x) = (pq, x^3 \bmod pq)$$

Cube root mod N assumption

Assumption:

For all poly-time A,

$$\Pr_{\substack{p,q \leftarrow \{0,1\}^\lambda \\ p,q \text{ prime}, N = pq \\ x \leftarrow \mathbb{Z}_N}} [A(N, x^3 \bmod N) = y : y^3 \equiv x^3 \bmod N] = \text{negl}(\lambda)$$

RSA Computational Problem

Cube root finding (believed hard): Given a, find x such that $x^3 \equiv a \pmod{N}$

How does this compare to square root finding? Depends on p, q

Tool: Euler's theorem. For all $a \in \mathbb{Z}_N^\times$ $(\gcd(a, N) = 1)$,

$$a^{\phi(N)} \equiv 1 \pmod{N}$$

where $\phi(N) = (p-1)(q-1) = |\mathbb{Z}_N^\times|$.

RSA Computational Problem

Cube root finding (believed hard): Given a, find x such that $x^3 \equiv a \pmod{N}$

How does this compare to square root finding? Depends on p, q

Tool: Euler's theorem. For all $a \in \mathbb{Z}_N^\times$ $(\gcd(a, N) = 1)$,

$$a^{\phi(N)} \equiv 1 \pmod{N}$$

Case 1: if 3 divides $\phi(N)$, then $x \mapsto x^3$ is many-to-one, and cube root-finding is as hard as factoring! (But it's not too different from square root finding.)

RSA Computational Problem

Cube root finding (believed hard): Given a, find x such that $x^3 \equiv a \pmod{N}$

How does this compare to square root finding? Depends on p, q

Tool: Euler's theorem. For all $a \in \mathbb{Z}_N^\times$ $(\gcd(a, N) = 1)$,

$$a^{\phi(N)} \equiv 1 \pmod{N}$$

Case 2: if 3 does not divide $\phi(N)$, then $x \mapsto x^3$ is a permutation on $\mathbb{Z}_N^\times$!

Reason: $\exists d: 3d \equiv 1 \pmod{\phi(N)}$. $y \mapsto y^d$ is an inverse map!

RSA Computational Problem

Cube root finding (believed hard): Given a, find x such that $x^3 \equiv a \pmod{N}$

How does this compare to square root finding? Depends on p, q

Tool: Euler's theorem. For all $a \in \mathbb{Z}_N^\times$ $(\gcd(a, N) = 1)$,

$$a^{\phi(N)} \equiv 1 \pmod{N}$$

Case 2: if 3 does not divide $\phi(N)$, then $x \mapsto x^3$ is a permutation on $\mathbb{Z}_N^\times$!

But: no longer know that cube root finding is as hard as factoring.

RSA Computational Problem

Candidate OWF:

$$f_N(x) = x^3 \bmod N$$

When 3 does not divide $(p-1)(q-1)$, this is a Trapdoor Permutation

- One-way permutation (defined on $\mathbb{Z}_N^\times$)
- Given trapdoor d, permutation becomes easy to invert!
- “RSA with exponent 3"

RSA Computational Problem

Another candidate trapdoor permutation:

$$f_{N,e}(x) = x^e \bmod N$$

where $\gcd(e, \phi(N)) = 1$. Typical: sample $e \leftarrow \mathbb{Z}^{\times}_{\phi(N)}$.

- "RSA trapdoor permutation"
- Given p, q, can find trapdoor d: $e \cdot d \equiv 1 \pmod{\phi(N)}$.
- $(x^e)^d \equiv x^{ed} \equiv x^1 \equiv x \bmod N$

RSA Assumption

Assumption:

For all poly-time A,

$$\Pr_{\substack{p,q\leftarrow\{0,1\}^\lambda \\ p,q \text{ prime}, N=pq \\ e\leftarrow\mathbb{Z}^\times_{\phi(N)} \\ x\leftarrow\mathbb{Z}^\times_N}} [A(N, x^e \bmod N) = x] = \text{negl}(\lambda)$$

Open: prove factoring assumption $\Leftrightarrow$ RSA assumption

RSA Assumption

Assumption:

For all poly-time A,

$$\Pr_{\substack{p,q\leftarrow\{0,1\}^\lambda \\ p,q \text{ prime}, N=pq \\ e\leftarrow\mathbb{Z}^\times_{\phi(N)} \\ x\leftarrow\mathbb{Z}^\times_N}} [A(N, x^e \bmod N) = x] = \text{negl}(\lambda)$$

Exercise: finding inverse $d \Leftrightarrow$ factoring.

“Textbook RSA Encryption”

Heuristic PKE scheme

$$\mathrm{Gen}(1^\lambda) \to (\mathrm{pk} = (N, e), \mathrm{sk} = d)$$
$$\mathrm{Enc}(\mathrm{pk}, m) = m^e \bmod N.$$
$$\mathrm{Dec}(\mathrm{sk}, \mathrm{ct}) = \mathrm{ct}^d \bmod N.$$

- Correct!
- Hard to recover random m given ct
- Deterministic, cannot satisfy any of our security definitions

“Textbook RSA Encryption”

Heuristic PKE scheme

$$\mathrm{Gen}(1^\lambda) \to (\mathrm{pk} = (N, e), \mathrm{sk} = d)$$
$$\mathrm{Enc}(\mathrm{pk}, m) = m^e \bmod N.$$
$$\mathrm{Dec}(\mathrm{sk}, \mathrm{ct}) = \mathrm{ct}^d \bmod N.$$

- Can be turned into a secure PKE in several ways (see HW3 for one way)
- Efficient CCA-secure variant of RSA used in practice (RSA-OAEP)

To be continued